中興大學 8113026207 科管博 薛慧婷 打造自己的 DNN(全連結)手寫辨識

一、作業目的與環境

用全連結神經網路 (DNN) 做 MNIST 手寫數字辨識，並比較不同架構、loss function、optimizer 對驗證/測試正確率的影響。實作環境為 Google Colab，最後用 Gradio 做成互動式手寫辨識展示介面。

開發環境 / 連結：

- Colab 連結：https://colab.research.google.com/drive/1JllBtSHSKK1AKTJZmi7qTWSupFrdFdwi?usp=sharing

二、模型架構設計

依照作業規定，隱藏層數不可為 3 層。

以下依序記錄「老師範例版（A 版）」與自行設計的改良版本（B、C 版）的架構、參數與結果。

2.1 老師範例版（A 版，作為對照基準）

架構：3 層隱藏層，每層 20 個神經元，activation 為 relu / relu / tanh，輸出層 softmax。

```
model.add(Dense(20, input_dim=784, activation='relu'))
model.add(Dense(20, activation='relu'))
model.add(Dense(20, activation='tanh'))
model.add(Dense(10, activation='softmax'))
```

Loss：mse　　Optimizer：SGD (learning_rate=0.087)

2.2 我的架構（最終採用版本）

為了避免跟範例過於相似，改用 5 層隱藏層（512 → 256 → 128 → 64），並在較寬的兩層後方加入 Dropout(0.3) 抑制 overfitting。輸出層維持 10 個神經元、softmax。

```
model = Sequential()
model.add(Input(shape=(784,)))
model.add(Dense(512, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(256, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(128, activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(10, activation='softmax'))
```

Loss：categorical_crossentropy　　Optimizer：Adam (learning_rate=0.001)

選擇 categorical_crossentropy 而非 mse 的原因：本題是多分類問題（10 個類別選 1 個），categorical_crossentropy 是專門設計來衡量「預測機率分布跟正確答案差多少」的 loss，mse 則是設計給連續數值（迴歸）問題使用，用在分類問題上收斂效果通常較差。

2.3 模型架構圖 (model.summary())

以下為最終採用架構（512-256-128-64-10）的 summary 輸出，可用來驗證各層參數量計算是否正確（例如第一層參數量 = 784 × 512 + 512 = 402,192）。

三、訓練設定

- batch_size：32（Keras 預設）
- epochs：10
- validation_split：0.1（切 10% 訓練資料當驗證資料，訓練過程中即時追蹤 val_accuracy）

加入 validation_split 的原因：驗證資料是模型訓練時完全沒看過的資料，驗證正確率比訓練正確率更能反映模型面對新資料的實力，這是挑選最終參數的依據，而不是只看訓練正確率。

四、實驗紀錄與結果比較

以下記錄實際跑過的每一組架構 / 參數設定與對應結果，包含表現最好的版本，以及刻意設計來驗證特定假設（如 loss function 的影響）的對照組。

4.1 各版本總覽

版本	隱藏層架構	Activation	Loss	Optimizer	測試正確率
A 版（老師範例）	20-20-20	relu/relu/tanh	mse	SGD(0.087)	94.55%
B 版（第一次修改，增加一層）	20-20-20-20	relu/relu/tanh/relu	mse	SGD(0.087)	92.72%
C 版（最終採用）	512-256-128-64	全 relu + Dropout(0.3)×2	categorical_crossentropy	Adam(0.001)	98.12%
D 版（控制變因對照組）	512-256-128-64（同 C）	同 C	mse	Adam(0.001)	97.82%

4.2 A 版（老師範例）、B 版（第一次修改）—— 訓練與結果

A 版：

Loss：0.0084　　測試正確率：94.55%

4. 檢視我們的神經網路

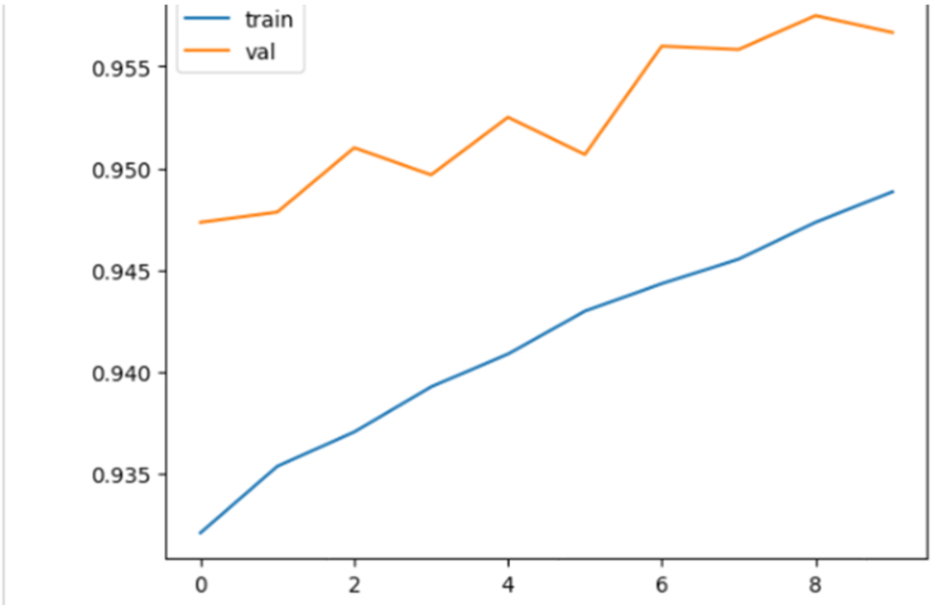
[22] 0 秒

```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_1 (Dense)	(None, 20)	15,700
dense_2 (Dense)	(None, 20)	420
dense_3 (Dense)	(None, 20)	420
dense_4 (Dense)	(None, 10)	210

Total params: 16,750 (65.43 KB)
Trainable params: 16,750 (65.43 KB)
Non-trainable params: 0 (0.00 B)




```
loss, acc = model.evaluate(x_test, y_test)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9455 - loss: 0.0084
```

```
print(f"測試資料正確率 {acc*100:.2f}%")
```

```
測試資料正確率 94.55%
```

觀察：A 版正確率明顯低於後續各版本，推測與使用 mse 作為分類問題的 loss function 有關（詳見 4.5 節分析）。

B 版　（第一次修改，擴增一層）

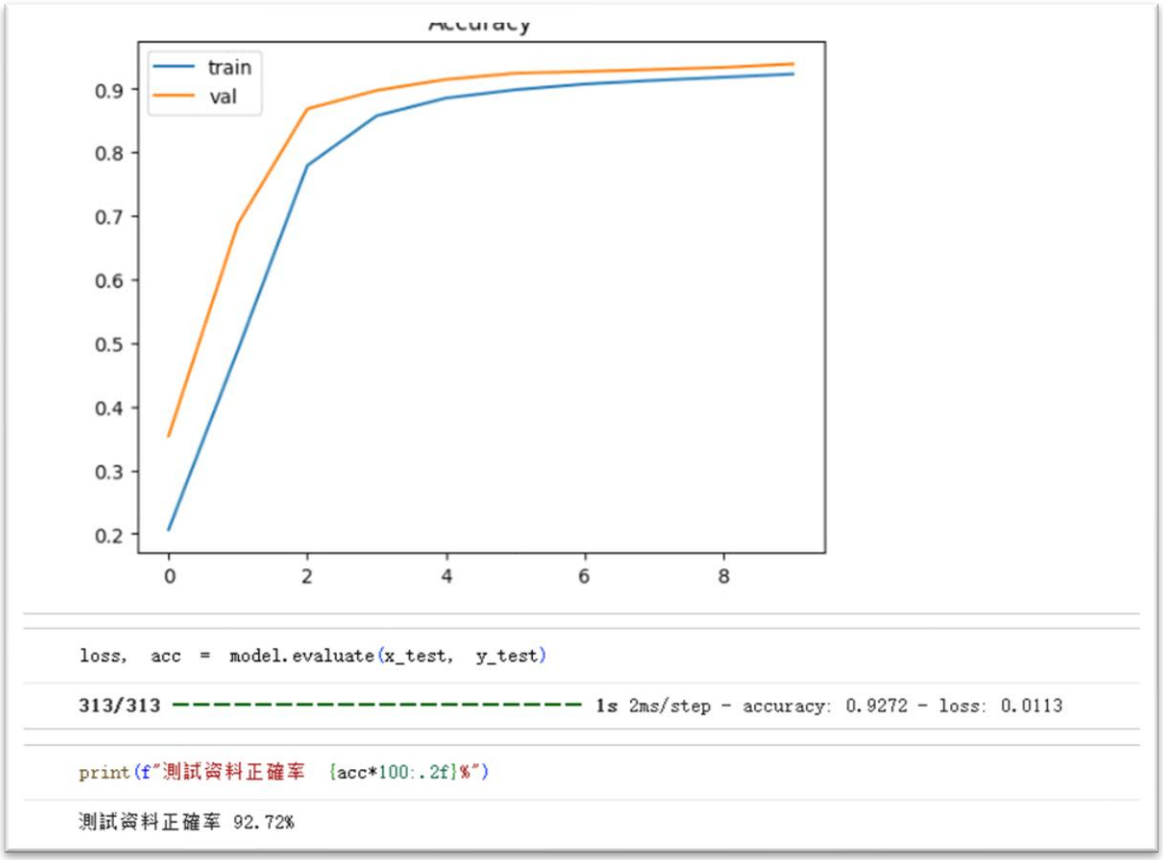
```
model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_11 (Dense)	(None, 20)	15,700
dense_12 (Dense)	(None, 20)	420
dense_13 (Dense)	(None, 20)	420
dense_14 (Dense)	(None, 20)	420
dense_15 (Dense)	(None, 10)	210

Total params: 17,170 (67.07 KB)
Trainable params: 17,170 (67.07 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 11s 6ms/step - accuracy: 0.2064 - loss: 0.0866 - val_accuracy: 0.3538 - val_loss: 0.0788
Epoch 2/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.4890 - loss: 0.0645 - val_accuracy: 0.6875 - val_loss: 0.0451
Epoch 3/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.7792 - loss: 0.0332 - val_accuracy: 0.8683 - val_loss: 0.0203
Epoch 4/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8576 - loss: 0.0216 - val_accuracy: 0.8975 - val_loss: 0.0159
Epoch 5/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8854 - loss: 0.0176 - val_accuracy: 0.9147 - val_loss: 0.0133
Epoch 6/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8986 - loss: 0.0155 - val_accuracy: 0.9245 - val_loss: 0.0119
Epoch 7/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9074 - loss: 0.0142 - val_accuracy: 0.9272 - val_loss: 0.0113
Epoch 8/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9131 - loss: 0.0133 - val_accuracy: 0.9303 - val_loss: 0.0108
Epoch 9/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9181 - loss: 0.0126 - val_accuracy: 0.9337 - val_loss: 0.0102
Epoch 10/10
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9232 - loss: 0.0119 - val_accuracy: 0.9390 - val_loss: 0.0095
驗證資料正確率最高出現在第 10 個 epoch，正確率為 93.90%
```



4.3 C 版（最終採用架構）—— 訓練與結果

Loss：0.06598　測試正確率：98.12%

4. 檢視我們的神經網路

```
#model.summary()
my_model.summary()
```

Model: "sequential_7"

Layer (type)	Output Shape	Param #
dense_19 (Dense)	(None, 512)	401,920
dropout_2 (Dropout)	(None, 512)	0
dense_20 (Dense)	(None, 256)	131,328
dropout_3 (Dropout)	(None, 256)	0
dense_21 (Dense)	(None, 128)	32,896
dense_22 (Dense)	(None, 64)	8,256
dense_23 (Dense)	(None, 10)	650

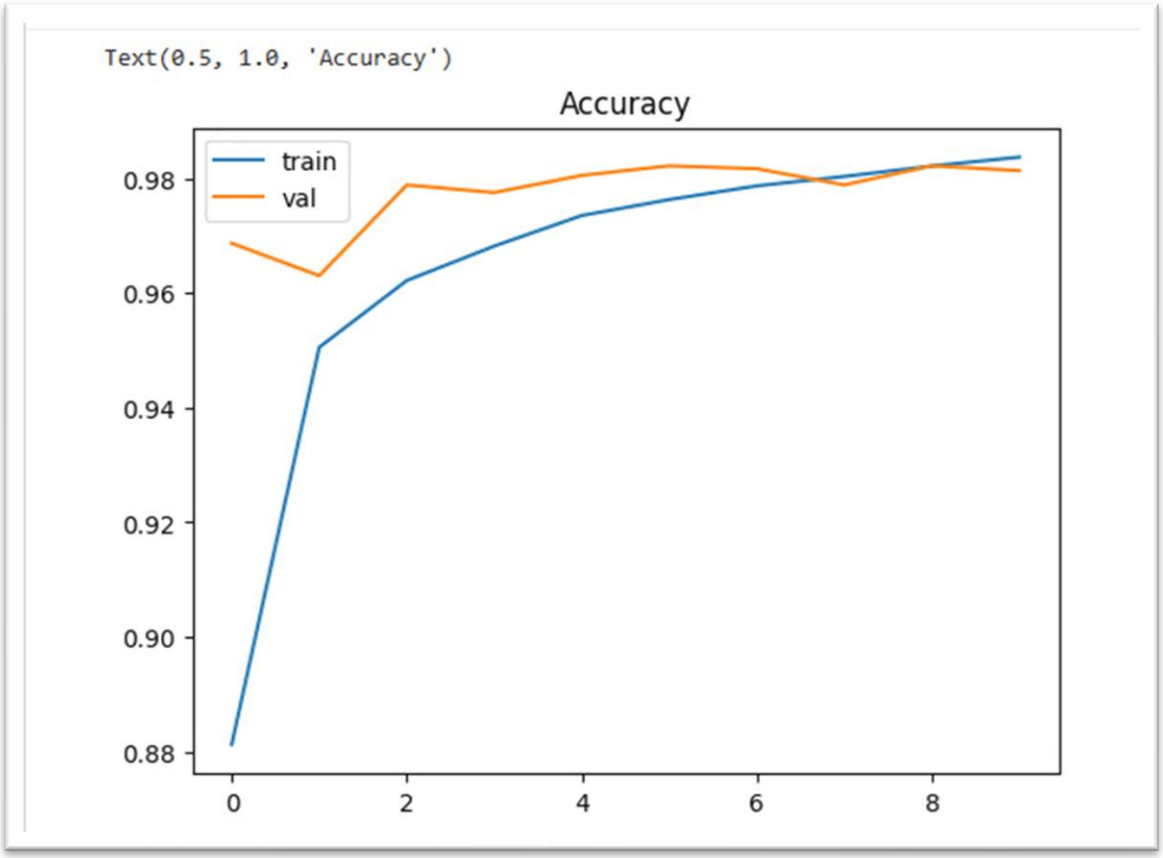
Total params: 575,050 (2.19 MB)
Trainable params: 575,050 (2.19 MB)
Non-trainable params: 0 (0.00 B)

```
score = model.evaluate(x_test, y_test)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9812 - loss: 0.0660
```

```
print('loss:', score[0])
print('正確率', score[1])
```

```
loss: 0.06598366796970367
正確率 0.9811999797821045
```




```
best_epoch = int(np.argmax(history.history['val_accuracy'])) + 1
best_val_acc = max(history.history['val_accuracy'])
print(f'驗證資料正確率最高出現在第 {best_epoch} 個 epoch，正確率為 {best_val_acc*100:.2f}%')
```

```
驗證資料正確率最高出現在第 8 個 epoch，正確率為 98.27%
```

觀察：val_accuracy 在第 8 個 epoch 達到最高（0.9827），第 9 個 epoch 反而小幅下降到 0.9792，第 10 個又回升到 0.9812。這種小幅震盪是正常現象，不算是 overfitting 的訊號。

4.4 D 版（控制變因對照組：架構同 C，但 loss 換回 mse）

設計目的：A、C 兩版同時改變了「架構」「loss function」「optimizer」三個變因，無法單獨判斷正確率差距的成因。因此設計 D 版：沿用 C 版架構與 optimizer，只把 loss 換回 mse，藉此單獨驗證「loss function 本身」對正確率的影響。

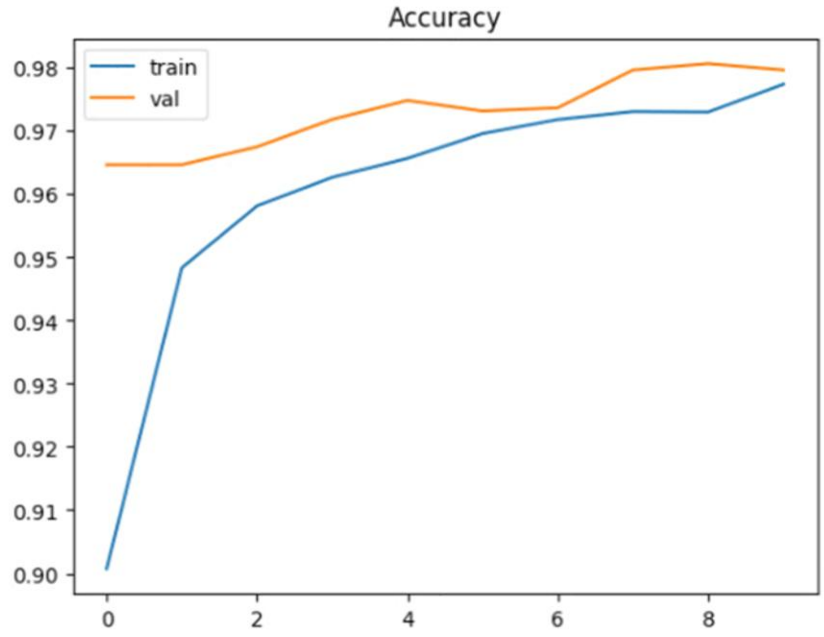
```
model.compile(loss='mse', optimizer=Adam(learning_rate=0.001),
metrics=['accuracy'])
```

Loss：0.0036%　測試正確率：97.82%

```
score = model.evaluate(x_test, y_test)
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9782 - loss: 0.0036

print('loss:', score[0])
print('正確率', score[1])
loss: 0.003552202135324478
正確率 0.9782000184059143
```

```
Text(0.5, 1.0, 'Accuracy')
```




```
loss, acc = model.evaluate(x_test, y_test)
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9782 - loss: 0.0036

print(f"測試資料正確率 {acc*100:.2f}%")
測試資料正確率 97.82%
```

結果解讀：

D 版正確率仍維持在 98% 左右：代表單純更換 loss function 不足以解釋 A、C 的差距，正確率差異可能主要來自 optimizer（SGD vs Adam）或架構容量（20-20-20 太小）等其他因素，需要再設計額外對照組驗證。

4.5 為什麼不能直接比較三版的 loss 數值？

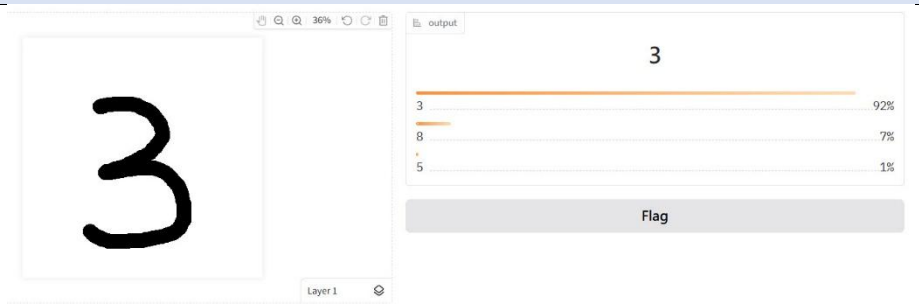
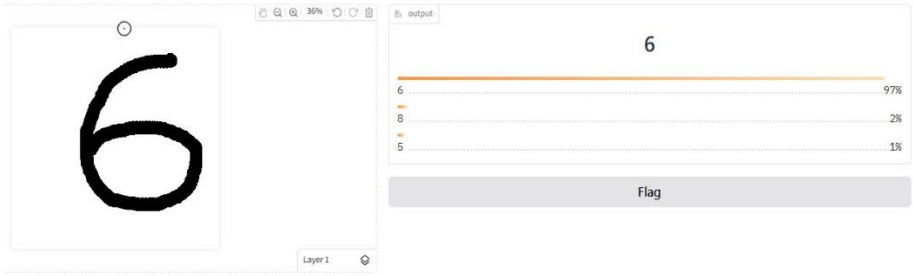
A 版（0.0084）、C 版（0.06598）、D 版（0.0036）三個 loss 數值放在一起看，容易誤以為「D 版 loss 最小，訓練得最好」，但這三個數字不是用同一把量尺量出來的：A、D 兩版用的是 mse，C 版用的是 categorical_crossentropy，不同 loss function 的計算方式跟數值尺度本來就不一樣，不能跨函數直接比大小。

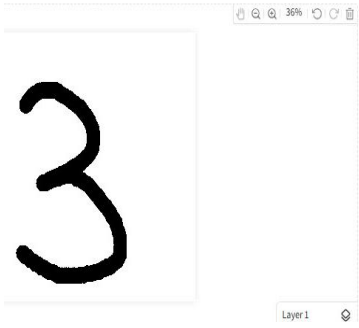
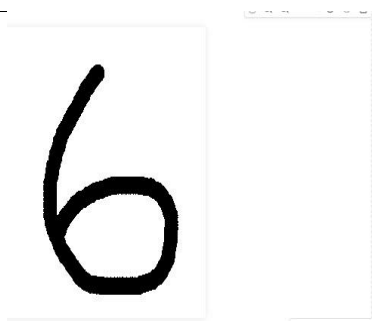
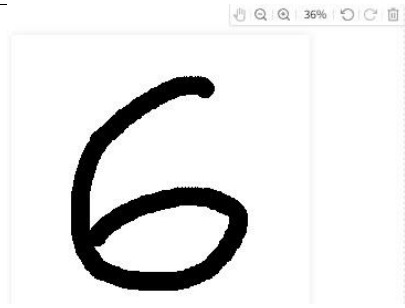
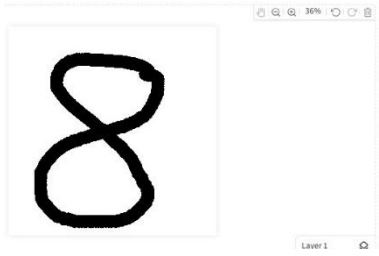
loss function 的選擇（mse vs categorical_crossentropy）在本次實驗中，對正確率的影響其實很小，真正拉開 A 版與其他版本差距的關鍵因素，是架構容量與 optimizer 的選擇，而不是 loss function 本身。

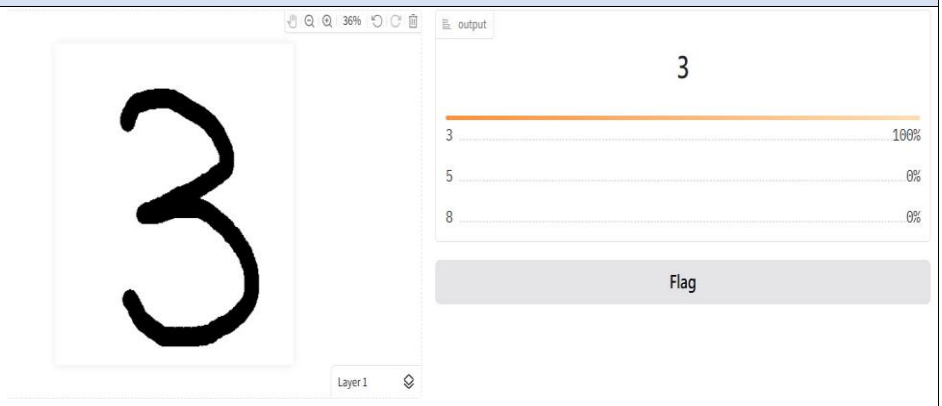
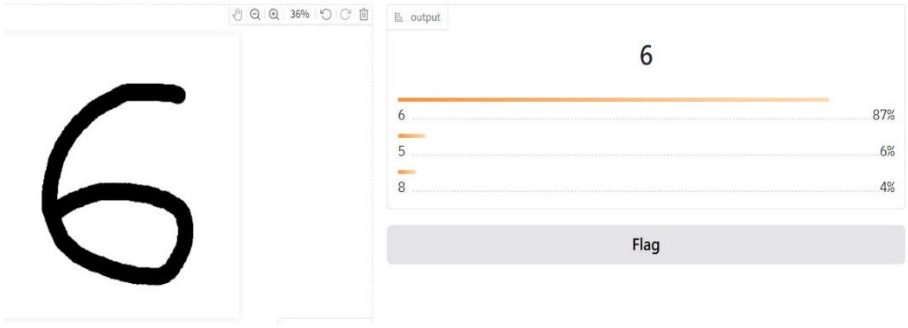
4.6 為什麼實際手寫測試（Gradio）正確率主觀感覺比 98% 低？

MNIST 訓練資料的筆畫特徵（偏粗、置中、灰階平滑漸層）跟使用者用滑鼠 / 觸控筆在 Gradio 畫板上手寫的風格（可能偏細、位置不置中、筆畫粗細不均）存在落差，這是典型的『訓練與部署資料分布不一致』（distribution shift）現象：測試資料正確率高，不代表模型能直接泛化到跟訓練資料分布不同的真實使用情境。

五、Gradio 展示

版本	判識率 (%)	實驗圖
A 版	92	
	97	
B 版	100	
	92	

版本	判識率 (%)	實驗圖
C 版	100	
	1	
C 版 (反覆實驗)	24	 

版本	判識率 (%)	實驗圖
D	100	
	87	

觀察：觀察實際手寫狀況與訓練集反應之正確率差異極大，查閱訓練資料集之圖片發現訓練資料中外國人士手寫文字有相當差異，另外，在版本C中【6及8】的判斷正確性較低，是未來可繼續強化之方向。

六、生成式 AI 使用說明

以下為本次作業實際使用 AI 協助的紀錄範本，請依實際對話內容填寫：

紀錄範本

用途	使用的 Prompt（節錄）	AI 回答摘要（自己的話）	我的理解與後續處理
架構設計建議	請求提供在原架構下模型改良的建議	提供版本 C 的方案及版本 4 做為比較條件	改成 5 層 512-256-128-64 並加 Dropout

用途	使用的 Prompt（節錄）	AI 回答摘要（自己的話）	我的理解與後續處理
程式執行問題	執行過程中發現若干執行程式 bug	AI 提供	1、 確認舊變數殘留於 Colab 記憶體，需重新啟動執行階段 2、 變數未宣告 3、 變數名稱前後不一

七、結論

1. 模型容量與 Optimizer 是效能關鍵

- 參數量影響：A 版（20-20-20）測試正確率為 94.55%，B 版（20-20-20-20）僅 92.72%；而擴大網路容量的 C 版（512-256-128-64）測試正確率顯著提升至 98.12%。單純加深窄網路（B 版）無法有效提升表現，充足的特徵擷取容量更為關鍵。

2. 資料分布落差（Distribution Shift）之限制與改善方向

- 模型在標準測試集具備 98% 左右的水準，但於 Gradio 介面實測時對特定手寫數字（如「6」與「8」）的辨識率明顯下滑。
- 數據分析顯示主因為使用者介面輸入與 MNIST 訓練集存在筆畫粗細、置中平滑度等分佈落差，未來可考慮進行輸入影像的預處理。